

- 1.没有过多协同开发经历
- 2.暂存区可以让用户精细地挑选本次要提交的改动，把不相关的修改留在工作区，避免一次性提交太多无关内容
- 3.git branch 只显示本地分支，git branch-a 显示所有分支，包括本地分支和远程分支

实验步骤：

首先对 main 分支修改为 feature 分支，修改代码后保存提交

然后修改回 main 分支，进行不同的修改后保存提交

此时合并两个分支导致了冲突

修改代码为一致后保存提交，解决冲突

```
ICS-GitLab > C main.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
6      <<<<<< HEAD (当前更改)
7      printf("hello,Fudan!HHH\n");// @TODO: print a sentence you
8      =====
9      printf("hello,FudanNNN!\n");// @TODO: print a sentence you
10     >>>>>> feature (传入的更改)
11     printf("Hello, world!\n");
12 }
```

```
ICS-GitLab > C main.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("hello,FudanNNN!\n");// @TODO: print a sentence you
6      printf("Hello, world!\n");
7  }
```

```
nanming@localhost:~/ics/ICS-GitLab$ git commit -m "解决冲突"
[main 01d0888] 解决冲突
nanming@localhost:~/ics/ICS-GitLab$ git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 22 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 504 bytes | 504.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects
.
```

选读文本：

1.Ruan 介绍了 Angular 提交规范，提交信息分为 Header、Body、Footer。Header 必填，格式为 `type(scope): subject`，type 包含 feat、fix 等七种类型。规范提交信息能方便快速浏览历史、过滤记录，并自动生成变更日志，这极大提升了团队协作与维护效率。

Gitflow 使用规范介绍了经典的分支管理模型，包含 master 和 develop 两个长期分支，以及 feature、release、hotfix 三个临时分支。feature 用于新功能，release 用于测试，hotfix 用于紧急修复线上 bug。核心原则是分支从哪里来最终就合并回哪里去，从而保证代码稳定和团队协作顺畅。